Adaptive Machine Learning Pipelines for Evolving Data Streams: A Comprehensive Review

Simran Shikha, Utsav Kumar(✉), Aastha Bansal, and Neha Kumari

Department of Computer Science and Engineering
Sharda School of Computing Science & Engineering, Sharda University Greater Noida, India
simranshikha96@gmail.com, utsav.udk@gmail.com, aasthabansal403@gmail.com, neha.kumari6@sharda.ac.in

Abstract. With the emergence of continuous and constantly changing data streams, the traditional machine learning pipeline, which is customarily built around static datasets and periodic retraining, shows clear limitations. Adaptive preprocessing, online feature selection, concept drift detection, incremental learning, and adaptive model updating are examples of adaptive machine learning techniques that help models follow varying data distributions. Existing systems tend to focus on predictive performance without considering latency or computational efficiency, which is problematic for real-time and resource-constrained environments. In this paper, we review key studies from the adaptive learning field, with emphasis on latency, adaptation mechanisms, and technologies. The included studies are compared in terms of the adaptations they use and their performance in prediction, processing efficiency, and deployment characteristics. We also conduct a comparative analysis to identify progress and gaps between predictive accuracy and latency for edge devices with limited memory and computational resources. The review highlights the need for an integrated, latency-sensitive adaptive machine learning framework that can switch its learning and processing approach dynamically as data and resource usage change. A multi-attribute comparison, an analysis of total adaptation cost, and a pipeline-stage analysis are provided. The proposed framework is a conceptual direction for future research and has not been experimentally validated; the reported results are specific to the individual studies and are not normalised comparisons.

Keywords: Adaptive Machine Learning · Evolving Data Streams · Concept Drift · Incremental Learning · Edge Computing · Latency · Predictive Accuracy · Streaming Analytics.

1 Introduction

IoT devices, sensor networks, financial platforms, and other real-time applications now produce a constant flow of evolving data. In contrast to static data, streaming data can show distribution drift and fundamental changes in data

patterns, known as concept drift, which can impair the performance of models that were trained only once [5, 7].

Adaptive machine learning allows models to keep adapting to such changes through techniques such as adaptive preprocessing, online feature selection, drift detection, and incremental learning [3–5, 19].

For real-time applications, however, predictive accuracy alone is not sufficient. Prediction latency, computational cost, memory consumption, and model update time also matter, especially when machine learning runs on edge devices. Edge-based learning has recently been studied in the contexts of adaptive stream analytics [11], resource-aware processing [12], concept-drift detection [13], and TinyML under resource constraints [14]. Recent techniques such as DFDT likewise show the need to balance prediction accuracy, speed, and memory usage on edge devices [15].

Despite this progress, current research tends to concentrate on separate aspects of accuracy, latency, adaptation, and resource efficiency rather than on the integrated pipeline. This review therefore examines original research on adaptive machine learning for evolving data streams, with a focus on technology, predictive performance, latency, and edge computing. It discusses the current state of the art together with its limitations and research gaps, and it motivates the development of an accuracy–latency-aware adaptive ML framework for data streams that evolve over time.

2 Background and Literature Review

Adaptive machine learning refers to the ability of a learning system to change its behaviour when the data change or when more data become available over time. Unlike conventional models trained on a fixed data set, adaptive models can be updated continuously and adjusted to changing data distributions. Several studies have addressed different aspects of adaptation: preprocessing, feature selection, model learning, and adaptation pipelines [3, 6, 18, 20].

Concept drift occurs when the underlying distribution or relationship in a data stream changes over time, which can degrade the predictions of a model that previously worked well. The adaptive windowing approach, such as ADWIN, detects statistically significant change and adjusts the window of historical data used for learning [5].

Incremental learning lets a model improve progressively as new observations arrive, without retraining on the whole data set. Adaptive online incremental learning has been studied for evolving data streams [18], and more recent continual and generalised incremental learning methods aim to retain learned knowledge in the presence of concept drift [1, 2]. These approaches can reduce the need for repeated full-scale training, but they may increase memory and computation requirements, depending on how often the model is updated.

TinyML aims to deploy machine learning on very resource-constrained devices with limited memory, processing power, and energy supply. For data

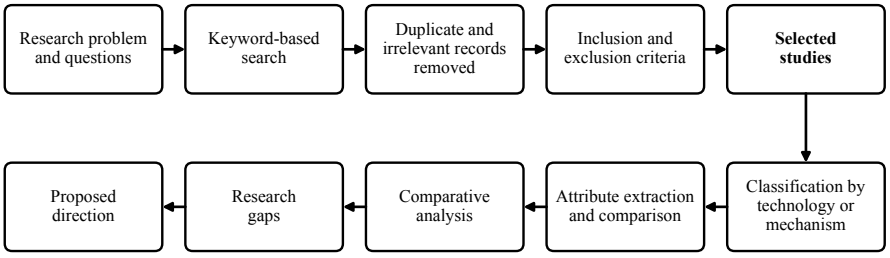


Fig. 1. Structured literature-review process used in this review

streams that change over time, TinyML raises additional challenges because adaptation has to take place within the limits of such hardware.

3 Review Methodology

This review follows a structured process to identify and analyse research on adaptive machine learning for evolving data streams. The process starts by defining the research problem and searching for relevant studies with keywords such as *adaptive machine learning, evolving data streams, concept drift, online learning, EdgeML, TinyML,* and *stream analytics*.

Duplicate and irrelevant publications were removed during screening. A study was included if it presents original research directly related to adaptive learning, evolving data streams, concept drift, incremental learning, streaming analytics, or resource-aware edge deployment. Studies that are mainly surveys or reviews, or that fall outside adaptive learning and evolving data streams, were excluded from the main analysis.

The final set comprises the 20 studies summarised in Table 1 [1–20]. Attributes that are not stated in the material available to this review were recorded as not reported (NR); no values were imputed or normalised. The review follows a structured search, screening, classification, and comparative-analysis procedure and is not presented as a fully PRISMA-compliant systematic review; database-level record counts are therefore not reported. The identification, screening, and analysis process is summarised in the flow shown in Fig. 1.

4 Comparative Analysis

The reviewed studies were compared mainly in terms of predictive accuracy, latency, and computational efficiency. The reviewed studies differ in datasets, hardware platforms, model configurations, deployment architectures, stream characteristics, and measurement protocols. Numerical accuracy and latency values are therefore not interpreted as normalised head-to-head comparisons; the comparison is treated as a qualitative, study-specific synthesis unless a common evaluation setting exists.

In the reviewed studies, the reported latency results are not measured in a consistent way. Thus, a common evaluation framework is needed to evaluate the accuracy and the latency under similar conditions. Latency is also reported at different granularity: per-instance runtime in [15] and total inference latency per deployment configuration in [11], which prevents direct conversion between the two.

4.1 Unified Multi-attribute Comparison

Table 1 consolidates the reviewed studies along common attributes: adaptation mechanism and update strategy, drift handling, reported predictive metric, latency or runtime, memory or resource consideration, hardware or deployment target, and edge/TinyML relevance. NR indicates that an attribute is not stated in the material available to this review; no value is imputed. The table is descriptive and does not rank the studies.

Table 1. Unified Comparison of Adaptive Machine Learning Approaches for Evolving Data Streams

Study (Year)	**Mechanism and drift handling**	**Reported accuracy, latency, and resource use**	**Deployment and main limitation**
Ashrafee et al. (2026) [1]	Continual learning with adaptive memory realignment; Concept drift; knowledge retention	Resource: Higher memory requirement	Limitation: Higher memory requirements
Yu et al. (2026) [2]	Generalised incremental learning; calibrated source-free adaptation; Joint evolution of distributions and label spaces	Accuracy: Superior performance reported; values NR; Latency: Training-free calibration avoids optimisation overhead	Limitation: Broader deployment evaluation needed
Lourenço et al. (2026) [15]	Dynamic fast decision tree for IoT stream mining	Accuracy: 66.7% average prequential accuracy (DFDTHigh); Latency: Per instance: 244.9 µs (DFDTHigh), 246.4 µs (DFDTLow), 241.5 µs (VFDT); Resource: Memory considered; values NR	IoT edge devices; platform NR; Edge/TinyML: Yes; limitation: Resource constraints create accuracy–cost trade-offs
Böther et al. (2025) [6]	Data-centric pipeline orchestration; automated data selection and retraining; Data-selection and triggering policies for retraining	Accuracy: Accuracy and training throughput reported; values NR	Limitation: Limited validation across heterogeneous streams
Hochma & Last (2025) [16]	Unsupervised online feature selection (OFFESEL)	Accuracy: Higher accuracy than compared methods on 17 streams; values NR; Latency: Less computation time than Max Variance (qualitative)	Limitation: Performance depends on data characteristics

Table 1. Unified Comparison of Adaptive Machine Learning Approaches for Evolving Data Streams (continued)

Study (Year)	**Mechanism and drift handling**	**Reported accuracy, latency, and resource use**	**Deployment and main limitation**
Niloy et al. (2025) [17]	Self-adaptive preprocessing (heuristics, statistics, reinforcement learning); Data-level (preprocessing) adaptation	Accuracy: Comparable accuracy reported; Resource: Human tuning time reduced by 85%; computational resources reduced by 22%	Limitation: Broader real-world evaluation required
Mehmood et al. (2024) [13]	Edge architecture; local drift detection and adaptive processing; Drift detection at the edge	Resource: Resource constraints remain	Edge (smart environments); device NR; Edge/TinyML: Yes; limitation: Resource and deployment constraints remain
Disabato & Roveri (2024) [14]	Deep feature extractor with k-NN; hybrid adaptation; On-device concept-drift adaptation	Accuracy: Effective on image and audio benchmarks; values NR; Resource: Limited memory and computation	Three off-the-shelf microcontroller units; Edge/TinyML: Yes; limitation: Limited computational and memory resources
Halstead et al. (2022) [7]	Probabilistic adaptation framework; Changing and recurring concepts	Accuracy: Recall and precision above 0.9 for selecting relevant concepts; Resource: Computational overhead noted; value NR	Limitation: Computational overhead
Wang et al. (2022) [11]	Edge-cloud integrated stream analytics; Pre-trained plus periodically re-trained LSTM under three drift scenarios	Accuracy: Dynamic approach best in accuracy; values NR; Latency: Total latency: 22.70 s (cloud), 17.08 s (edge), 16.64 s (edge-cloud)	Cloud, edge, and edge-cloud deployments; Edge/TinyML: Yes (edge-cloud); limitation: Deployment complexity
Petri et al. (2022) [12]	Resource-aware edge stream analytics; processing adapted to available resources	Resource: Adapts to available resources	Local and remote edge resources; testbed; Edge/TinyML: Yes; limitation: Constrained edge resources remain challenging
Zhang et al. (2021) [18]	Autoencoder with memory module; online incremental learning; Reconstruction-loss-based drift detection triggers model update	Accuracy: Outperforms state-of-the-art methods; values NR	Limitation: Adaptation affected by abrupt drift
Celik & Vanschoren (2021) [8]	Adaptive AutoML; six concept-drift adaptation strategies; Adaptation strategies on evolving data	NR	Limitation: Limited preprocessing integration
Montiel et al. (2021) [10]	River: online ML framework (Python)	Resource: Lightweight stream learning (qualitative)	Python library; limitation: Limited distributed capability
Gupta & Hewett (2019) [19]	Adaptive normalisation; Changing feature distributions	Accuracy: RMSE of 0.0041 on 160,000 instances; 89% improvement over baseline	Apache Storm (distributed); limitation: Mainly numerical features
Gomes et al. (2017) [20]	Adaptive random forest (ensemble); Classification under drift	Accuracy: 85.29% average accuracy (ARF-fast); Resource: Complexity increases	Limitation: Increased computational complexity

Table 1. Unified Comparison of Adaptive Machine Learning Approaches for Evolving Data Streams (continued)

Study (Year)	**Mechanism and drift handling**	**Reported accuracy, latency, and resource use**	**Deployment and main limitation**
Žliobaitė & Gabrys (2014) [3]	Adaptive preprocessing; Preprocessing adapted to changing streams	NR	Limitation: Limited large-scale evaluation
Morales (2013) [9]	SAMOA: distributed stream-mining platform	Resource: Requires distributed infrastructure	Distributed infrastructure; limitation: Requires distributed infrastructure
Hoi et al. (2012) [4]	Online feature selection	Resource: Dimensionality reduction	Limitation: Parameter-sensitive
Bifet & Gavaldà (2007) [5]	ADWIN adaptive windowing; Statistical change detection; window adaptation	NR	Limitation: Affected by noisy streams

NR: not stated in the material available to this review; no value is imputed. Values are study-specific and are not normalised across studies.

The comparison in Table 1 shows which dimensions are commonly and rarely evaluated. Every study is characterised by its adaptation mechanism and by a qualitative limitation, but predictive metrics appear quantitatively for only a small subset of the summarised studies ([7, 15, 19, 20]), and latency or runtime for fewer still ([11, 15]). Memory is mostly described qualitatively, and energy consumption is not reported as a separate quantity for any entry in the material summarised here.

Model-adaptation studies ([1, 2, 7, 18, 20]) emphasise how a learner retains or recovers accuracy under drift, and their limitations concern memory, computation, or abrupt drift. Edge-oriented studies ([11–15]) emphasise resources and deployment: [11] reports latency per deployment configuration, [15] reports per-instance runtime together with prequential accuracy, and [12–14] describe resource-aware or on-device adaptation. Preprocessing, feature-selection, and orchestration studies ([3, 4, 6, 8, 16, 17, 19]) address the configuration or automation of pipeline stages rather than edge resource use.

Accuracy, latency, memory, and adaptation cost are therefore evaluated separately: [20] reports accuracy, [11] reports deployment-level latency, and [15] reports accuracy and runtime for a decision-tree family but at the model stage only. In the material summarised here, no entry reports the cost of preprocessing, feature selection, drift detection, and model update as parts of one end-to-end measure. This is why a unified evaluation, in which accuracy, latency, resource use, and adaptation cost are measured under the same conditions, remains necessary.

4.2 Critical Synthesis by Component

Adaptive preprocessing [3, 17, 19]. These studies adapt preprocessing or normalisation to changing stream characteristics [3, 19] and automate preprocessing

operations [17]. They assess robustness of learning to changing inputs, but the cost of updating preprocessing state per instance and its share of end-to-end latency are not reported in the material summarised here. For the proposed direction, preprocessing should be treated as a configurable and costed stage that a controller can simplify when latency budgets are exceeded.

Online feature selection [4, 16]. These studies reduce dimensionality and unnecessary computation while preserving predictive performance [4, 16]. They measure retained performance and selection computation, but not how a changed feature set alters downstream update and inference time on constrained devices, nor how selection interacts with drift detection. The size of the active feature set is therefore a natural control variable that trades accuracy against computational cost.

Concept drift detection [5, 7, 13, 14]. Adaptive windowing [5], probabilistic handling of changing and recurring concepts [7], and edge or on-device detection [13, 14] establish how change can be identified. They measure the ability to detect and adapt to change, but not jointly the detection delay, the detection overhead, and the resulting accuracy, nor how the detection signal coordinates other stages. Drift detection is thus best treated as the trigger that supplies a drift status to a controller, with its own overhead counted.

Incremental and ensemble learning [1, 2, 18, 20]. These studies retain or recover accuracy without full retraining [1, 2, 18, 20]. They measure predictive performance, but update time, memory growth, and energy for repeated updates are not reported separately in the summarised material. This implies that model-update time and model size must be reported, and that a lightweight adaptation mode is needed when resources are scarce.

Pipeline orchestration, AutoML, and streaming frameworks [6, 8–10]. These works automate data selection and retraining [6], model adaptation [8], distributed stream mining [9], and lightweight stream learning [10]. They measure automation and scalability rather than execution under constrained resources. Orchestration decisions should therefore be linked to the resource state of the target device.

Edge and resource-aware approaches [11–15]. These studies address edge-cloud deployment [11], resource-aware analytics [12], edge drift detection [13], TinyML adaptation [14], and edge decision trees [15]. They measure deployment latency [11] and runtime and memory trade-offs [15], but not a complete adaptive pipeline, including preprocessing, feature selection, and model update, under changing resources. This motivates adaptation that responds jointly to drift and to resource state.

Taken together, adaptive preprocessing reduces the cost associated with changing input characteristics, while online feature selection reduces unnecessary feature computation. Drift detection provides the trigger for adaptation, and incremental learning updates the model. However, these components are frequently evaluated separately rather than as one end-to-end adaptive pipeline, and the reviewed evidence does not show how their costs combine.

4.3 Evaluation and Adaptation Cost

Latency and efficiency values in the reviewed studies were measured under heterogeneous hardware, datasets, and deployment settings. Normalising such values would require assumptions that the source studies do not support; this review therefore does not normalise them. Instead, it specifies the items that a future common evaluation protocol should report. **Hardware.** Processor architecture, CPU frequency or class, RAM, accelerator (GPU or NPU) where applicable, and edge device category. **Dataset.** Dataset type, number of features, stream length, and drift characteristics (drift type and frequency). **Model.** Model architecture, number of parameters, and model size. **Latency.** Preprocessing latency, feature-selection latency, drift-detection latency, inference latency, model-update latency, retraining latency, and total end-to-end pipeline latency, reported separately and preferably as a distribution (mean and tail percentiles) rather than a single average. **Resource.** CPU utilisation, memory usage, energy consumption, storage, and communication overhead. Until such a protocol is adopted, the values in Table 1 should be read as study-specific rather than as normalised head-to-head benchmarks.

Inference latency alone does not describe the cost of an adaptive system. This review distinguishes three quantities. **Inference latency** is the time to produce a prediction with the current model. **Adaptation latency** is the time consumed by the operations that keep the pipeline current: preprocessing updates, online feature-selection updates, drift detection, and incremental model updates (model-update time). **Retraining latency** is the time needed to rebuild or substantially revise the model, in full or on a buffer of recent data (retraining overhead).

The total adaptation cost is defined here conceptually as the sum of these components, each of which has a time, memory, and energy footprint:

$$C_{\text{total}} = C_{\text{pre}} + C_{\text{fs}} + C_{\text{drift}} + C_{\text{inf}} + C_{\text{upd}} + C_{\text{ret}} + C_{\text{comm}} \tag{1}$$

A system with low inference latency can still have a high system-level cost if it triggers expensive updates or retraining frequently, because the cost per update accumulates with the update frequency, which depends on drift frequency and detector sensitivity. On memory-limited devices, buffers and ensembles increase memory use [1, 20], and on energy-limited devices repeated updates can shorten operating time. Resource-aware and TinyML studies [12, 14, 15] consider memory, runtime, or computational limits, but update and retraining phases are not reported separately as latency or energy quantities in the summarised material. This omission is carried into the research gaps of Section 5 and into the metrics of the proposed direction in Section 6.

Table 2 summarises the computational role of each pipeline stage. It does not rank the stages: which stage dominates end-to-end latency depends on feature dimensionality, stream rate, drift frequency, the algorithm, model complexity, hardware, and adaptation frequency.

Table 2. Pipeline-Stage Computational and Adaptation Considerations

Pipeline stage	**Function and adaptation role**	**Potential computational cost and latency impact**
Adaptive preprocessing	Adjusts scaling, normalisation, or cleaning to changing stream statistics [3, 19]; keeps the input representation consistent with the current distribution	Per-instance work; grows with the number of features and the complexity of adaptation rules; adds a small, recurring delay to every instance
Online feature selection	Selects or weights features by relevance [4, 16]; adjusts the active feature set as relevance changes	Depends on the selection algorithm and dimensionality; may reduce downstream cost; selection adds time; a smaller feature set may shorten update and inference
Concept drift detection	Monitors errors or distributions to signal change [5, 7, 13, 14]; provides the trigger for adaptation	Per-instance or per-window monitoring; detector-dependent; adds monitoring time; detection delay affects timeliness of adaptation
Incremental learning	Updates the model per instance or batch [18, 20]; absorbs new data without full retraining	Depends on model type, ensemble size, and update frequency; model-update time added per update; ensembles multiply it
Model retraining	Rebuilds the model on buffered or selected data [6]; recovers from severe or recurring drift	Potentially the highest per event; depends on buffer size and model; may delay or interrupt predictions unless run in the background
Inference	Produces a prediction with the current model; produces predictions and accuracy feedback	Depends on model complexity; direct contribution to prediction latency

The reviewed studies rarely attribute end-to-end cost to individual stages: [15] reports per-instance runtime and [11] reports total latency per deployment configuration, but neither, as summarised here, decomposes cost by stage. The differential effect of online feature selection and adaptive preprocessing on end-to-end latency therefore remains an open question, and it motivates the stage-level metrics in Section 6.

5 Research Gaps

Although the reviewed studies demonstrate continued development in adaptive machine learning for evolving data streams, several research gaps remain.

One of the drawbacks is that there is not much integration between the pipeline stages. Most methods focus on a specific aspect, whereas only few studies present a single mechanism that can address the different aspects of preprocessing, feature selection, drift detection, model adaptation and continuous monitoring [3, 6, 18].

Another gap that is important is the consideration of predictive accuracy and latency together. Latency, runtime, and computational efficiency are not evaluated using consistent metrics across the reviewed studies [6, 11, 12, 20].

The comparative analysis of Section 4 exposes further gaps concerning measurement and cost. Latency and resource use are measured inconsistently across studies, and on heterogeneous hardware and deployment settings, so reported values cannot be normalised (Section 4.3). Model-update and retraining overhead are seldom reported as separate latency, memory, or energy quantities, and

stage-level attribution of end-to-end cost is rare, which leaves the differential effect of adaptive preprocessing, online feature selection, drift detection, and model update on end-to-end latency largely unquantified (Section 4.3). In addition, drift-driven adaptation and resource-driven adaptation are usually treated separately, and complete adaptive pipelines are rarely evaluated under edge constraints.

These gaps are connected. Without a unified benchmarking and evaluation protocol, the accuracy–latency trade-off cannot be assessed jointly across pipeline stages, and without stage-level and update-level measurements, the total adaptation cost cannot be attributed. Table 3 lists the resulting gaps, from limited pipeline integration to the lack of unified evaluation. They motivate the conceptual framework of Section 6, which treats drift status and resource status as joint triggers of adaptation and defines the metrics needed to address them.

Table 3. Research Gaps Identified from the Reviewed Studies

Research gap	**Existing work**	**Identified gap**
Pipeline Integration	Adaptive preprocessing [3], incremental learning [18], and pipeline orchestration [6] address different stages of adaptation.	Limited integration of multiple adaptive components into a unified pipeline.
Latency and Efficiency	Edge-cloud analytics [11] and resource-aware stream analytics [12] address processing efficiency.	Latency is not consistently evaluated together with predictive accuracy across approaches.
Hardware and Deployment Heterogeneity	Studies use cloud, edge, edge-cloud, and constrained-device settings [11–15].	Results from heterogeneous hardware and deployment settings cannot be compared without a common protocol.
Resource-Aware Adaptation	Resource-aware edge approaches [12], TinyML [14], and DFDT [15] consider memory, runtime, or computational limitations.	Limited dynamic adaptation based simultaneously on data drift and available system resources.
Update and Retraining Overhead	Incremental updates [18], continual learning [1, 2], and orchestrated retraining [6] maintain predictive performance.	Limited analysis of model-update and retraining latency, memory, and energy.
Stage Bottleneck Attribution	Individual stages, such as preprocessing [3], feature selection [4, 16], and drift detection [5], are studied separately.	Limited attribution of end-to-end cost to individual pipeline stages.
Unified Evaluation	Existing studies use different datasets, metrics, and experimental environments [6, 7, 11–15].	Difficult to directly compare accuracy, latency, and efficiency across existing methods.

6 Proposed Research Direction

The identified research gaps motivate a conceptual, integrated adaptive machine learning framework as a research direction for evolving data streams with a focus on predictive accuracy, latency and resource efficiency.

The proposed research direction is to investigate an accuracy–latency-aware adaptive machine learning framework for edge-based streaming environments. The framework will include adaptive preprocessing, concept drift detection, incremental learning and online feature selection to facilitate the continuous adaptation of models. Furthermore, system-level monitoring will be included to track

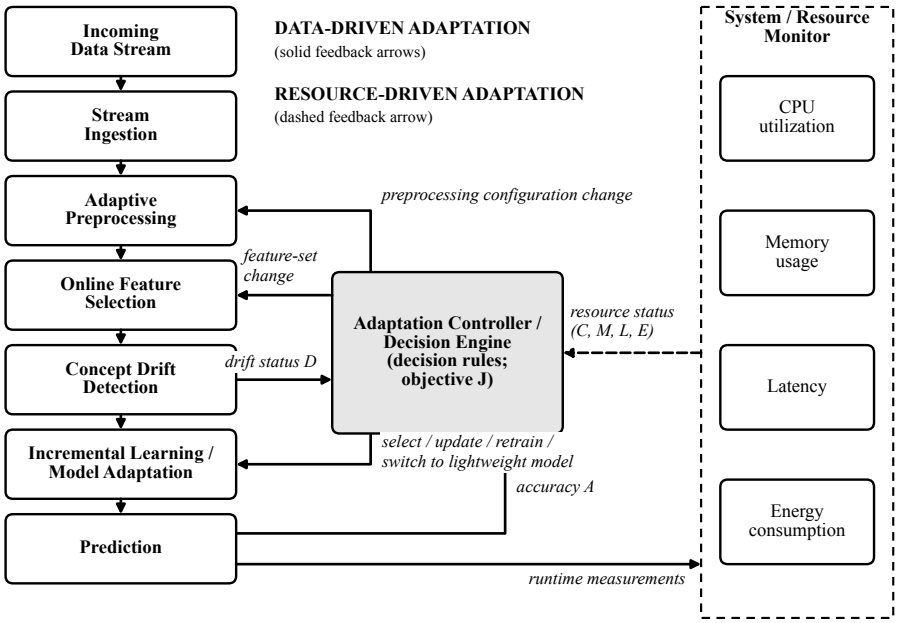


Fig. 2. Conceptual architecture of the proposed accuracy–latency-aware adaptive machine learning framework for evolving data streams. Solid feedback arrows denote data-driven adaptation; the dashed arrow denotes resource-driven adaptation. The framework is a research direction and has not been experimentally validated.

system resource usage, processing time, and prediction performance, during operation.

6.1 Conceptual Architecture and Workflow

Fig. 2 shows the intended architecture. Data pass through stream ingestion, adaptive preprocessing, online feature selection, concept drift detection, and incremental learning or model adaptation before a prediction is produced. A system and resource monitor observes CPU utilisation, memory usage, latency, and energy consumption during operation. An adaptation controller (decision engine) receives the drift status D and the predictive accuracy A from the data path, and the resource status (C, M, L, E) from the monitor. It returns control actions: a change of preprocessing configuration, a change of the active feature set, model selection or switching to a lightweight model, an incremental update, or retraining.

Data-driven adaptation responds to changes in the stream: the drift status D and the observed accuracy A indicate whether the current pipeline configuration still matches the data. Resource-driven adaptation responds to system state: latency L, computational cost C, memory utilisation M, and energy consumption

E indicate whether the pipeline can sustain its configuration on the target device. Considering both is the central difference from the reviewed approaches, which treat drift adaptation [5, 7, 18] and resource awareness [12, 14, 15] largely separately (Section 5).

6.2 Adaptation Policy and Decision Rules

Let A denote predictive accuracy (or F1-score) over a sliding window, L latency, C computational cost (corresponding to C_{total} in Section 4.3), M memory utilisation, E energy consumption, and D the drift status. Application-defined thresholds $A_{\min}$, $L_{\max}$, $C_{\max}$, $M_{\max}$, and $E_{\max}$ express the requirements of a deployment; resources are regarded as sufficient when $C \leq C_{\max}$, $M \leq M_{\max}$, and $E \leq E_{\max}$. The controller would evaluate the following conceptual rules on each monitoring window.

R1. IF concept drift is detected **AND** resource availability is sufficient **THEN** perform incremental model adaptation.
R2. IF concept drift is detected **AND** memory or CPU constraints are severe **THEN** use a lightweight adaptation strategy.
R3. IF $L > L_{\max}$ **THEN** reduce preprocessing complexity, reduce the feature set, or switch to a lower-complexity model.
R4. IF $A < A_{\min}$ **AND** resources are available **THEN** initiate stronger model adaptation or retraining.
R5. IF $M > M_{\max}$ **THEN** reduce model or buffer complexity, or switch to a resource-efficient configuration.

When rules conflict, for example when accuracy has fallen below $A_{\min}$ while memory exceeds $M_{\max}$, resource limits are treated as hard constraints, and stronger adaptation is deferred or replaced by a lightweight alternative until resources allow. These rules are conceptual mechanisms. Their thresholds, ordering, and effect on accuracy and latency have not been validated in this review and are subject to the planned experiments.

6.3 Multi-objective Formulation

The controller is intended to select a pipeline configuration θ (preprocessing setting, active feature subset, model, and update strategy) that minimises the objective

$$J = \alpha(1 - A) + \beta L + \gamma C + \delta M + \varepsilon E \tag{2}$$

subject to $L \leq L_{\max}$, $M \leq M_{\max}$, and $E \leq E_{\max}$. Here $A \in [0, 1]$ is predictive accuracy over a sliding window; L, C, M, and E are normalised by application-specific reference values so that the terms are commensurate; and α, β, γ, δ, and ε are non-negative, application-dependent weights. This is a proposed optimisation formulation for future experimental validation; it is not a result of the current review, and the choice of weights, the normalisation, and the search over configurations remain open design questions.

6.4 Planned Evaluation

In future work, the proposed framework will be evaluated on evolving data-stream datasets and compared with existing adaptive learning approaches under the common reporting protocol of Section 4.3. The planned metrics are accuracy, F1-score, prediction latency, end-to-end pipeline latency, drift-detection latency, model-update time, retraining time, CPU utilisation, memory usage, energy consumption, computational overhead, and, where relevant, communication overhead.

7 Conclusion

This review examined adaptive machine learning approaches for evolving data streams, covering adaptive preprocessing, online feature selection, concept drift detection, incremental learning, streaming frameworks, pipeline orchestration, and edge-based learning. The comparative analysis reveals that existing methods are able to handle the changing data distribution, but the accuracy, latency and resource utilization are not comprehensively measured simultaneously. The unified comparison and the component-wise synthesis show that the reviewed studies differ in datasets, hardware, and measurement protocols, so their numerical values are study-specific rather than normalised head-to-head results. Model-update and retraining overhead and stage-level attribution of end-to-end cost are rarely reported.

The review highlights that limited integration is seen throughout adaptive pipeline stages and no common evaluation settings for predictive and system-level performance. These gaps motivate a conceptual adaptive ML framework that considers data drift and resource constraints jointly through an adaptation controller, decision rules, and a multi-objective formulation; the framework has not been experimentally validated.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ashrafee, A., Kozal, J., Woźniak, M., Krawczyk, B.: Holistic Continual Learning under Concept Drift with Adaptive Memory Realignment. Trans. Mach. Learn. Res. (2026)
2. Yu, E., Lu, J., Zhang, G.: Generalized incremental learning under concept drift across evolving data streams. In: Proceedings of the ACM Web Conference (WWW), pp. 3905–3916 (2026). https://doi.org/10.1145/3774904.3792379
3. Žliobaitė, I., Gabrys, B.: Adaptive preprocessing for streaming data. IEEE Trans. Knowl. Data Eng. **26**(2), 309–321 (2014). https://doi.org/10.1109/TKDE.2012.147
4. Hoi, S.C.H., Wang, J., Zhao, P., Jin, R.: Online feature selection for mining big data. In: Proceedings of the 1st International Workshop on Big Data, Streams and Heterogeneous Source Mining (BigMine), pp. 93–100 (2012). https://doi.org/10.1145/2351316.2351329

5. Bifet, A., Gavaldà, R.: Learning from time-changing data with adaptive windowing. In: Proceedings of the 7th SIAM International Conference on Data Mining (SDM), pp. 443–448 (2007). https://doi.org/10.1137/1.9781611972771.42
6. Böther, M., et al.: Modyn: Data-centric machine learning pipeline orchestration. Proc. ACM Manag. Data **3**(1), 1–30 (2025). https://doi.org/10.1145/3709705
7. Halstead, B., Koh, Y.S., Riddle, P., Pechenizkiy, M., Bifet, A.: A probabilistic framework for adapting to changing and recurring concepts in data streams. In: Proceedings of the IEEE 9th International Conference on Data Science and Advanced Analytics (DSAA), pp. 1–10 (2022). https://doi.org/10.1109/DSAA54385.2022.10032368
8. Celik, B., Vanschoren, J.: Adaptation strategies for automated machine learning on evolving data. IEEE Trans. Pattern Anal. Mach. Intell. **43**(9), 3067–3078 (2021). https://doi.org/10.1109/TPAMI.2021.3062900
9. Morales, G.D.F.: SAMOA: A platform for mining big data streams. In: Proceedings of the 22nd International Conference on World Wide Web Companion, pp. 777–778 (2013). https://doi.org/10.1145/2487788.2488042
10. Montiel, J., et al.: River: Machine learning for streaming data in Python. J. Mach. Learn. Res. **22**(110), 1–8 (2021)
11. Wang, X., Khan, A., Wang, J., Gangopadhyay, A., Busart, C.E., Freeman, J.: An edge-cloud integrated framework for flexible and dynamic stream analytics. Future Gener. Comput. Syst. **137**, 323–335 (2022). https://doi.org/10.1016/j.future.2022.07.023
12. Petri, I., Chirila, I., Gomes, H.M., Bifet, A., Rana, O.F.: Resource-aware edge-based stream analytics. IEEE Internet Comput. **26**(4), 79–88 (2022). https://doi.org/10.1109/MIC.2022.3152478
13. Mehmood, H., Khalid, A., Kostakos, P., Gilman, E., Pirttikangas, S.: A novel Edge architecture and solution for detecting concept drift in smart environments. Future Gener. Comput. Syst. **150**, 127–143 (2024). https://doi.org/10.1016/j.future.2023.08.023
14. Disabato, S., Roveri, M.: Tiny machine learning for concept drift. IEEE Trans. Neural Netw. Learn. Syst. **35**(6), 8470–8481 (2024). https://doi.org/10.1109/TNNLS.2022.3229897
15. Lourenço, A., Rodrigo, J., Gama, J., Marreiros, G.: DFDT: Dynamic fast decision tree for IoT data stream mining on edge devices. Proc. AAAI Conf. Artif. Intell. **40**(29), 24053–24060 (2026). https://doi.org/10.1609/aaai.v40i29.39583
16. Hochma, Y., Last, M.: Fast online feature selection in streaming data. Mach. Learn. **114**(1), 1 (2025). https://doi.org/10.1007/s10994-024-06712-x
17. Niloy, D.B., Jony, M.E.H., Ruhani, H.K.: A self-adaptive data preprocessing pipeline for machine learning: An automated and dynamic approach. In: Proceedings of the International Conference on Quantum Photonics, Artificial Intelligence, and Networking (QPAIN), pp. 1–6 (2025). https://doi.org/10.1109/QPAIN66474.2025.11171989
18. Zhang, S.S., Liu, J.W., Zuo, X.: Adaptive online incremental learning for evolving data streams. Appl. Soft Comput. **105**, 107255 (2021). https://doi.org/10.1016/j.asoc.2021.107255
19. Gupta, V., Hewett, R.: Adaptive normalization in streaming data. In: Proceedings of the 3rd International Conference on Big Data Research (ICBDR), pp. 12–17 (2019). https://doi.org/10.1145/3372454.3372466
20. Gomes, H.M., et al.: Adaptive random forests for evolving data stream classification. Mach. Learn. **106**(9–10), 1469–1495 (2017). https://doi.org/10.1007/s10994-017-5642-8